1.1.4. Minimum S... 22:41



Write a C program to construct the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph using Kruskal's Algorithm.

You are provided with a driver code. Complete the function **kruskalMST()** so that the program correctly finds and prints the Minimum Spanning Tree and its total cost. The graph is represented using an adjacency matrix.

Implement the function:

```
void kruskalMST(int **cost, int V)
```

Where,

- $V \rightarrow$ Number of vertices in the graph
- $\text{cost} \rightarrow$ Adjacency matrix storing edge weights

Input Format:

- The first line of input reads an integer V by prompting "No of vertices: ", representing the number of vertices in the graph.
- The next $V \times V$ integers represent the adjacency matrix of the graph with a heading "Adjacency matrix:"
- $\text{cost}[\text{i}][\text{j}]$ indicates the weight of the edge between vertex i and vertex j
- If there is no edge between two vertices, the weight should be given as 9999 (representing infinity).
- The graph is undirected, so the matrix is symmetric

Output Format:

Explorer

C minCostF...

Submit

Debugger

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

int find(int parent[], int i){
    if(parent[i]!=i){
parent[i]=find(parent,parent[i]);
    }
    return parent[i];
}
void union1(int parent[], int rank[], int u, int v) {
    int rootU=find(parent, u);
    int rootV=find(parent, v);

    if (rootU != rootV){
        if(rank[rootU]<rank[rootV]){
parent[rootU]=rootV;
        }
        else if(rank[rootU]>rank[rootV]){
parent[rootV]=rootU;
        }else{
parent[rootV]=rootU;
rank[rootU]=rank[rootU]+1;
        }
    }
    // Write your code
```

‹ Prev Reset Submit Next ›

1.The output should display each edge selected for the Minimum Spanning Tree in the order they are chosen in the below format:

```
Edge k:(u, v) cost:w
```

Where:

- k is the edge number starting from 0
- u and v are the vertices connected by the edge
- w is the weight of the edge

2.In the last line the program should print the total minimum cost of the MST as follows:

```
Minimum cost= X
```

Where X is the total cost of the Minimum Spanning Tree.

Constraints:

- $1 \leq V \leq 100$
- Edge weights are non-negative integers
- The adjacency matrix is dynamically allocated
- Exactly $V - 1$ edges must be included in the MST

Note:

- Process the edges in ascending order of their weights. For edges with the same weight, follow their row-wise order in the upper triangular part of the adjacency matrix.
- Refer to the visible test cases to strictly match the input and output layout.

Sample Test Cases +

```
}


void kruskalMST(int **cost, int V) {

    int parent[V],rank[V];
    int mst_weight=0;
    int edge_count=0;

    for(int i=0; i<V;i++){
        parent[i]=i;
        rank[i]=0;
    }
    while(edge_count<V-1){
        int min=9999;
        int u=-1, v=-1;
         int i,j;
        for(i=0;i<V;i++){
            for(int j=0; j<V; j++){
if(cost[i][j] !=0 && find(parent, i)!=find(parent,j)){
if(cost[i][j]<min){
min=cost[i][j];
u=i;
v=j;
                    }
                }
            }
        }
        if(u!=-1 && v!=-1 ){
```

```
union1(parent,rank ,u,
v);

printf("Edge %d:(%d,
%d)
cost:%d\n",edge_count,
u, v,min);

mst_weight=mst_weight+mi
n;

edge_count=edge_count+1;
        }

    }
    printf("Minimum
cost= %d\n",mst_weight);
}


int main() {
    int V;
    printf("No of
vertices: ");
    scanf("%d", &V);

    int **cost = (int
**)malloc(V *
sizeof(int *));
    for (int i = 0; i <
V; i++)
        cost[i] = (int
*)malloc(V *
sizeof(int));

    printf("Adjacency
matrix:\n");
    for (int i = 0; i <
V; i++)
        for (int j = 0;
j < V; j++)
            scanf("%d",
```